数学与 CS 问答笔记

math-answer-note

按日归档的问答笔记：三问反思法（关键点/变式/同类归类）+ 讲课叙事流 + CeTZ 图示，结构与课程笔记同构。

October 01, 2026

Contents

1. 为什么包名改成 db 了，还是报 undefined: KV 4
   1.1. 费曼直觉 4
   1.2. 硬核机制 4
      1.2.1. 实测：这个模块里有三个包，其中两个都叫 db 4
      1.2.2. 报错到底来自哪个文件 4
      1.2.3. 反向验证：往那个目录里塞什么都会 undefined 5
      1.2.4. 两个包对照 5
   1.3. 陷阱与易错 5
   1.4. 关键点 6
   1.5. 变式检验 6
   1.6. 同类归类 6
   1.7. Go 语言机制补充 6
      1.7.1. 包名、目录名、导入路径三者的关系 6
      1.7.2. 为什么 Go 允许包名与目录名不一致 7
   1.8. 本次操作与当前状态 7
      1.8.1. 做过的操作 7
      1.8.2. 当前状态 7
      1.8.3. 待办 7
2. 补全 KV 的三个方法：先确认语义，再动手 8
   2.1. 费曼直觉 8
   2.2. 硬核机制 8
      2.2.1. 第一步：在课程材料里定位语义 8
      2.2.2. 第二步：补全实现 9
      2.2.3. 第三步：验证（四步全绿） 10
   2.3. 陷阱与易错 10
   2.4. 关键点 10
   2.5. 变式检验 10
   2.6. 同类归类 11
   2.7. Go 语言机制补充 11
      2.7.1. map 的键为什么必须“可比较” 11
      2.7.2. 命名返回值与裸 return 11
   2.8. 本次操作与当前状态 11
      2.8.1. 做过的操作 11
      2.8.2. 当前状态 12
      2.8.3. 待办 12
3. 回顾：undefined: KV 到底是在哪里修好的 13
   3.1. 费曼直觉 13
   3.2. 硬核机制 13
      3.2.1. 三次 undefined: KV，三次不同的根因 13
      3.2.2. 我上一步实际做的五件事 13
      3.2.3. 下次照着走的排查清单 14

3.3. 当前状态：你把实现清空了（练习模式） 14
3.4. 关键点 15
3.5. 变式检验 15
3.6. 同类归类 15
3.7. Go 语言机制补充 15
3.7.1. nil map 为什么读安全、写危险 15
4. 怎么用 go doc 查标准库的用法与实现 16
4.1. 费曼直觉 16
4.2. 硬核机制 16
4.2.1. 常用形态速查 16
4.2.2. 四个实例，看清差异 16
4.2.3. 三种包都能查 17
4.3. 陷阱与易错 18
4.4. 关键点 18
4.5. 变式检验 18
4.6. 同类归类 18
4.7. Go 语言机制补充 19
4.7.1. 文档就是源码注释 19
4.7.2. `go doc` 背后的实现 19
Index of Listings 20

1. 为什么包名改成 db 了，还是报 undefined: KV

1.1. 费曼直觉

因为决定“是不是同一个包”的**不是包名，而是目录**。

你把 `kv_test.go` 放进 `test/` 目录、第一行写 `package db` ——这不等于“加入了 `internal/db` 那个 db 包”，而是创建了**另一个**也叫 `db` 的包，只不过它住在 `test/` 目录里。两个包名字相同、内容毫无关系，就像两栋楼都挂了“3 单元”的牌子，但一栋在 A 街、一栋在 B 街。

`KV` 定义在 `internal/db/kv.go` 里，属于 A 街那个包；你在 B 街的包里喊 `KV`，当然查无此名。

1.2. 硬核机制

1.2.1. 实测：这个模块里有三个包，其中两个都叫 db

```
$ go list -f '{{.ImportPath}}  包名={{.Name}}' ./...
github.com/zz-cpp/db-go/cmd/dbshell  包名=main
github.com/zz-cpp/db-go/internal/db  包名=db
github.com/zz-cpp/db-go/test         包名=db
```

Listing 1: 同一份输出里两个包的 Name 都是 db，但 ImportPath 不同——它们就是两个包

`go list` 报表里的 `ImportPath` 才是包的身份，`Name` 只是“写代码时用哪个前缀”。

1.2.2. 报错到底来自哪个文件

你贴的位置 `[10, 8]` 精确指向这里：

```
$ go vet ./test/
vet: test/kv_test.go:10:8: undefined: KV

$ gopls check test/kv_test.go
test/kv_test.go:10:8-10: undefined: KV
```

Listing 2: 报错文件是 `test/kv_test.go` —— 不是 `internal/db/kv_test.go`

第 10 行第 8 列，正是 `kv := KV{}` 里 `KV` 的起始位置。而**同一时刻**，同样内容的 `internal/db/kv_test.go` 却毫无问题：

```
$ gopls check internal/db/kv_test.go
（无输出）

$ gopls check internal/db/kv.go
（无输出）
```

Listing 3: 同机实测：放对目录的那一份，gopls 一个字都不报

两份文件内容**完全相同**（`diff test/kv_test.go internal/db/kv_test.go` 无差异），唯一的区别是所在目录——结果却一个满屏红叉、一个干干净净。

1.2.3. 反向验证：往那个目录里塞什么都会 undefined

为了排除“是不是 kv_test.go 自己写错了”，我在 `test/` 目录里新建了一个探针文件：

```
package db

var _ = KV{}   // 试图引用 KV
```

Listing 4: 探针：在 test/ 目录里引用 KV

```
$ go build ./test/
# github.com/zz-cpp/db-go/test
test/zz_probe.go:3:9: undefined: KV
```

Listing 5: 照样 undefined——因为那个目录下根本没有 KV 的声明

1.2.4. 两个包对照

对比项	`internal/db/` 里的包	`test/` 里的包
导入路径	`github.com/zz-cpp/db-go/internal/db`	`github.com/zz-cpp/db-go/test`
包名（Name）	`db`	`db`（一样！）
目录里有什么	16 个实现文件 + 9 个测试文件	只有一份 `kv_test.go`
`KV` 在哪	就在这里定义（`kv.go`）	不在，所以 undefined
结果	能编译、能跑测试	只能报错

> 📌 **核心要点** 记住这条不变量：**包 = 目录**。包名只是访问时用的前缀，**不参与身份判定**。所以“把包名写成 db”永远不会让你加入别的包——只有“把文件放进那个目录”才会。

1.3. 陷阱与易错

1. **以为包名相同就是同一个包**。包身份由导入路径（即目录位置）决定，包名撞车也互不干扰；
2. **编辑器 buffer 把已删除的文件写回来**。`test/kv_test.go` 是今天 14:12 重新出现的，而它昨天已被移走并删除——很可能是 nvim 里仍开着旧 buffer，保存时把文件又写了回去。删掉文件之后记得把对应 buffer 也关掉（`:bd`），否则下次保存它还会长出来；
3. **在错误目录里调试**。判断“我在哪个包里”最快的办法是 `go list -f '{{.ImportPath}}' .`，或直接看编辑器状态栏；
4. **拿“内容相同”当“位置无所谓”**。这两份文件字节完全一致，但一份能跑一份报错——位置本身就是语义。

1.4. 关键点

> 📌 **核心要点** `undefined: KV` 不一定意味着“还没实现”，也可能是“你看错目录了”。判断顺序：① 报错文件的路径是什么；② 它所在目录里有没有这个符号的声明；③ 那个目录是不是被测包所在的目录。

1.5. 变式检验

1. **换成同一目录里写两个包名**：会直接报 `found packages db and test in /path` ——一个目录只能有一个普通包名；反过来，**不同目录用同一个包名完全合法**（本例就是，编译器不会给任何警告）；
2. **换成 `go test ./test/`**：报错与 `go vet` 一致，因为测试二进制同样要对这个包做类型检查；
3. **换成从 `test/` 包导入真正的 db 包**：那就成了普通跨包引用——`import "github.com/zz-cpp/db-go/internal/db"` 加 `db.KV{}`。那是“两个包互相引用”，与“放在同一个包下”是两回事。

1.6. 同类归类

1. 与 Python 的对照：Python 的包身份同样是“目录（+ `__init__.py`）”，模块名由路径决定；文件放错目录，`import` 照样找不到——与 Go 完全同构；
2. 与 Java 的对照：Java 里包名与目录结构强绑定（`package com.example.db` 必须在 `com/example/db/` 下），编译器会直接拒绝错位的文件；Go 更宽松——包名与目录名**可以**不一致（`internal/db` 目录里就叫 `db`），代价是“同名不同目录”不会报错，只能靠自己分辨；
3. 往下延伸：这个坑在真实项目里会以“同名包”的形式反复出现（多个项目都有 `util`、`config`），老手判断包的依据永远是**导入路径**——看代码先看 import 段，而不是先看包名。

1.7. Go 语言机制补充

1.7.1. 包名、目录名、导入路径三者的关系

概念	由谁决定	作用
导入路径	模块路径 + 目录相对路径，由 `go.mod` 与文件位置共同决定	包的**身份**，唯一标识一个包
目录名	文件系统里的名字（如 `internal/db`）	只影响导入路径的后半段
包名	每个 `.go` 文件首行的 `package xxx` 声明	代码里的**访问前缀**，如 `db.KV`

三者可以任意错位而互不影响，于是出现这些**合法**情形：

1. 目录 `internal/db` / 包名 `db` ——本仓库（目录名带组织信息，包名取便于书写的部分）；
2. 目录 `yaml` / 包名 `yaml` ——最常见的一一对应；

3. 目录 `go-yaml` / 包名 `yaml` ——目录名里有 `-`，而连字符不是合法标识符，包名必须另取；
4. 目录 `test` / 包名 `db` ——今天这一例：合法，但语义上误导人。

1.7.2. 为什么 Go 允许包名与目录名不一致

因为导入路径要“表达层级与来源”，包名要“简短好写”。像 `internal/db`、`gopkg.in/yaml.v3`、`github.com/user/repo/v2/pkg` 这类路径，末段要么带组织信息、要么带版本号，都不适合直接当前缀。于是 Go 把两件事分开：**路径负责定位，包名负责书写**。点导入（`import . "..."`）能生效，也是同一个道理——它只是在改“书写形式”。

> 📌 **核心要点** 一句话收束：**看包先看路径，不看到包名。** `go list -f '{{.ImportPath}} {{.Name}}' ./...` 一条命令同时给出两者，是排查“同名包”问题最快的工具。

1.8. 本次操作与当前状态

这一小节记录本次排查**实际做了什么、现在卡在哪**——不只记知识，也记动作，方便日后回看。

1.8.1. 做过的操作

操作	细节
删除 `test/` 目录	里面的 `kv_test.go` 与 `internal/db/kv_test.go` 内容完全相同（`diff` 无差异），属重复文件
新建本日期文件	`note/2026-10-01.typ`，使用完整头部（从 `2026-09-30.typ` 复制前 130 行）
修复一处静默渲染错误	caption 里两处裸写的 `test/kv_test.go` 被 Typst 配成下标语法，已包进 raw

1.8.2. 当前状态

```
$ go build ./...
# github.com/zz-cpp/db-go/internal/db
internal/db/kv.go:14:6: missing function body
internal/db/kv.go:15:6: missing function body
internal/db/kv.go:16:6: missing function body
```

Listing 6: 当前唯一阻塞：kv.go 里 Get / Set / Del 三个方法还只有签名

顺带印证上一节（`go vet 通过 ≠ 能编译`）：此刻 `go vet ./...` 是**干净**的，而 `go build ./...` 报上面三处——同一条命令集，两种结论。

1.8.3. 待办

1. 在 `internal/db/kv.go` 里给 `Get`、`Set`、`Del` 补上函数体；
2. 编辑器里若还开着 `test/kv_test.go` 的 buffer，用 `:bd` 关掉——否则保存时会把已删除的文件重新写出来（这份文件正是今天 14:12 以这种方式重新出现的）。

2. 补全 KV 的三个方法：先确认语义，再动手

2.1. 费曼直觉

这类问题的本质是：**接口契约写在测试里，语义细节却常常没写。**

`kv_test.go` 把签名、返回值的顺序都说清楚了（`Set` 返回 `(bool, error)`、`Del` 返回 `(bool, error)`），但那个 `bool` 到底表示什么，测试只覆盖了一半：

1. `Del` 的语义被测得明明白白——删不存在的键返回 `false`，删存在的键返回 `true`；
2. `Set` 的语义**完全没有被测**——测试只调了一次 `Set`，且是新键。也就是说“重复写入同一个值时该返回 true 还是 false”，测试给不出答案。

遇到这种情况，**不要猜**：去找权威来源。这个仓库的 README 里写着讲义在 `../db-web/`，而去那里一查，课程作者连每一步的完整解答都附上了。

2.2. 硬核机制

2.2.1. 第一步：在课程材料里定位语义

```
    $ ls /home/zz/project/db-web/web/trialofcode.org/database/
    0101_kv_interface/   0203_update_mode/  0401_sort_search/  ...

    $ ls /home/zz/project/db-web/web/trialofcode.org/database/db_project+solution/
db_solution/
    0101  0102  0103  ...  0704   ← 每一步的「参考解答」都在这里

    $ ls .../db_solution/0101/
    kv.go  kv_test.go
```

Listing 7: 讲义目录结构：既有课程正文，也有每一步的参考解答

顺着这条线索就拿到了课程作者的第 1 步实现（`db_solution/0101/kv.go`），关键三行是：

```
    func (kv *KV) Get(key []byte) (val []byte, ok bool, err error) {
      val, ok = kv.mem[string(key)]
      return
    }

    func (kv *KV) Set(key []byte, val []byte) (updated bool, err error) {
      prev, exist := kv.mem[string(key)]
      kv.mem[string(key)] = val
      updated = !exist || !bytes.Equal(prev, val)
      return
    }

    func (kv *KV) Del(key []byte) (deleted bool, err error) {
      _, deleted = kv.mem[string(key)]
      delete(kv.mem, string(key))
      return
    }
```

Listing 8: 课程作者的 0101 实现——`updated` 的真实语义在这里：值真的变了才为 true

而那份 `kv_test.go` 与本仓库里的测试文件**逐字相同**（只差包名），所以它就是这套 API 的标准答案。

> 📌 **核心要点** `updated` 的准确含义是**“值真的发生了变化”**：新键算变化；老键写入不同的值算变化；**写入一模一样的值返回 false**。这行 `updated = !exist || !bytes.Equal(prev, val)` 是三个方法里唯一有“设计感”的地方——它让调用方能区分“我这次写入产生了实际效果”与“数据库本来就是这个样子”。

2.2.2. 第二步：补全实现

按同样的语义补进 `internal/db/kv.go`（包名用本仓库的 `db`）：

```
package db

import "bytes"

type KV struct {
  mem map[string][]byte
}

func (kv *KV) Open() error {
  kv.mem = map[string][]byte{} // empty
  return nil
}

func (kv *KV) Close() error { return nil }

func (kv *KV) Get(key []byte) (val []byte, ok bool, err error) {
  val, ok = kv.mem[string(key)]
  return
}

func (kv *KV) Set(key []byte, val []byte) (updated bool, err error) {
  prev, exist := kv.mem[string(key)]
  kv.mem[string(key)] = val
  updated = !exist || !bytes.Equal(prev, val)
  return
}

func (kv *KV) Del(key []byte) (deleted bool, err error) {
  _, deleted = kv.mem[string(key)]
  delete(kv.mem, string(key))
  return
}
```

Listing 9: 补全后的 kv.go：新增 import “bytes”，三个方法各就各位

四处值得留意的细节：

1. **`[]byte` 不能做 map 的键**。Go 规定 map 的键必须是**可比较类型**，而切片（含 `[]byte`）不可比较。所以这里统一用 `string(key)` 转换；

2. **那次转换是零拷贝**。Go 为 `string(b)` 在 map 查找/赋值这种场景开了特例：编译器不会真的复制字节（规范与实现在 `mapaccess` / `mapassign` 系列里有专门优化）。所以不必担心“为了用 map 而多复制一份数据”；
3. **比较字节要用 `bytes.Equal`**，不能用 `==`。`[]byte` 上的 `==` 只有在和 `nil` 比较时才有意义，两个切片之间比较会编译失败；
4. **`delete` 对不存在的键是安全空操作**。所以 `Del` 得先查一次拿 `deleted`，再删——顺序反了就拿不到“删没删到”这个信息了。

2.2.3. 第三步：验证（四步全绿）

```
$ gofmt -l .          # 无输出
$ go build ./...      # 通过
$ go vet ./...        # 通过
$ go test -v ./...
?     github.com/zz-cpp/db-go/cmd/dbshell [no test files]
=== RUN   TestKVBasic
--- PASS: TestKVBasic (0.00s)
PASS
ok    github.com/zz-cpp/db-go/internal/db 0.003s
```

Listing 10: 四个命令全部通过，测试转绿

> 📌 **核心要点** 修完 `missing function body` 后，编译、vet、测试三关要**同时过**——前面几节反复印证过：`go vet` 干净不代表能编译，`go build` 通过也不代表测试能跑。四步都跑一遍才叫验完。

2.3. 陷阱与易错

1. **凭感觉猜语义**。`Set` 返回的布尔值，看名字会以为是“是否是新键”，实际是“值是否变化”——两者对重复写入的行为完全不同。项目里有现成的权威材料时，查一下比猜快得多；
2. **给 `Set` 直接返回 `true`**。那样测试照样通过（因为没测重复写入），但语义就错了，调用方再也分不清“写了”和“白写”；
3. **用 `==` 比较字节切片**。编译直接报错：`invalid operation: prev == val (slice can only be compared to nil)`；
4. **忘了 `import "bytes"`**。加了 `bytes.Equal` 就会用上，漏写会报 `undefined: bytes`。

2.4. 关键点

> 📌 **核心要点** 实现一个接口时，**签名由编译器把关，语义由测试和文档把关**。测试没覆盖到的语义（本例中 `Set` 的返回值），必须回到权威材料确认——课程解答、官方文档、或参考实现，任选其一都比猜可靠。

2.5. 变式检验

1. **换成“重复写入不同值”**：`Set` 会返回 `true`，因为是真正的更新；

2. **换成“重复写入相同值”**：返回 `false` ——这一条正是测试没覆盖、而语义规定死了的分支，值得自己补一个测试用例（见下一节）；
3. **换成删除后再查**：`Del` 之后 `Get` 返回 `ok=false`，测试最后三行正是在验证这个闭环。

2.6. 同类归类

1. 与 `map` 的对照：这层的 `KV` 本质就是“给 `map[string][]byte` 包一层方法”，`Open` / `Close` 是为了将来换成磁盘存储时接口不变——课程从内存起步正是这个用意（讲义第一课就解释了 in-memory 与 disk-based 的关系）；
2. 与数据库的对照：`Set` 返回“是否真的改变”这一设计，和 Redis 的 `SET ... NX/XX`、以及磁盘存储里“这次写入是否产生了新的日志记录”是同一类信息，越往后越重要——它决定了能否跳过写盘；
3. 往下延伸：课程下一步会给 `Set` 加上 `UpdateMode`（只看新值、只更新已存在的键），那时这个布尔值的含义会再次被细化。

2.7. Go 语言机制补充

2.7.1. map 的键为什么必须“可比较”

Go 规范要求 map 的键类型必须是**可比较**（comparable）的：切片、map、函数都不可比较，所以不能做键；而 `string`、整数、指针、以及由它们构成的数组/结构体可以。实践中最常见的需求就是“用字节序列做键”——标准做法就是 `string(b []byte)` 转换，代价通常被编译器优化掉。

2.7.2. 命名返回值与裸 return

三个方法都用了“命名返回值 + 裸 `return`”的写法：

```
1    func (kv *KV) Get(key []byte) (val []byte, ok bool, err error) {
2      val, ok = kv.mem[string(key)]
3      return   // 等价于 return val, ok, err
4    }
```

Listing 11: 命名返回值让“给哪个变量赋值”一目了然，裸 return 省去重复列举

好处是函数签名自己就是文档（`val`、`ok`、`err` 三个名字说明了返回值含义），且短函数里读起来更干净。缺点是长函数里裸 `return` 会让人忘了当前值是什么——所以 Go 社区的建议是“要么函数短，要么别用裸 return”。

> 📌 **核心要点** 一句话收束：**签名告诉你“传什么收什么”，语义告诉你“返回值意味着什么”**。前者编译器盯着，后者只能靠测试、文档或参考实现——测试没覆盖的分支，一定要去查，不要自己发明。

2.8. 本次操作与当前状态

2.8.1. 做过的操作

操作	细节

补全 `internal/db/kv.go`	新增 `import "bytes"`；`Get` / `Set` / `Del` 补上函数体，语义取自课程解答 `db_solution/0101/kv.go`
查证而非猜测	到 `../db-web/` 找到课程作者的第 1 步解答，确认 `updated` 表示“值真的变了”

（`kv_test.go` 本轮未改动——它与课程作者的测试逐字相同，本来就正确。）

2.8.2. 当前状态

```
$ gofmt -l .      # 无输出
$ go build ./...  # 通过
$ go vet ./...    # 通过
$ go test ./...   # ok  github.com/zz-cpp/db-go/internal/db  0.003s
```

Listing 12: 第 1 步的 KV 完成，四步全绿

2.8.3. 待办

1. 可选：给 `Set` 补一个“重复写入相同值应返回 `false`”的测试用例——这是当前测试没覆盖、而语义已经定死的分支；
2. 编辑器里若还开着 `test/kv_test.go` 的 buffer，用 `:bd` 关掉（原因见上一节）。

3. 回顾：undefined: KV 到底是在哪里修好的

3.1. 费曼直觉

直接回答：**我一个字都没改 `kv_test.go`** —— `undefined: KV` 的报错位置在那里，但根因从来不在那里。

这是能反复救命的一条心法：**报错位置 ≠ 问题位置**。Go 的名字解析错误永远标在“使用处”，而问题往往藏在“声明处”或“包的边界上”。在每个报错的文件里就地瞎改，是最容易浪费半天的做法。

3.2. 硬核机制

3.2.1. 三次 undefined: KV，三次不同的根因

同一个报错、同样的位置 `[10, 8]`，背后的原因完全不同：

序号	报错文件	真正的根因	修的是哪里
1	`test/kv_test.go`	文件声明 `package test`，而 `KV` 属于 `db` 包——跨包访问要带前缀并 import	`kv_test.go`（加 import + `db.` 前缀）
2	`test/kv_test.go`	包名改对了，但 `internal/db/kv.go` 里根本没有 `KV`——骨架项目还没写实现	`kv.go`（补 `type KV struct` 及其方法）
3	`test/kv_test.go`	文件声明 `package db`，却住在 `test/` 目录——**包由目录决定**，那是另一个包	目录结构（去掉 `test/`，文件回到 `internal/db/`）

三次里，`kv_test.go` 的内容只在第 1 次真正需要改（加前缀）；第 2、3 次的修改都发生在别处。

3.2.2. 我上一步实际做的五件事

1. **确认报错来自哪个文件**：`go vet ./test/` 与 `gopls check test/kv_test.go` 的输出里带着完整路径——`test/kv_test.go:10:8`。先看清是哪个 `kv_test.go`（当时仓库里有两份）；
2. **定位根因**：`go list -f '{{.ImportPath}} 包名={{.Name}}' ./...` 打出三个包，其中 `main`、`db`、`db`——两个名字都叫 `db` 但导入路径不同，说明它们的身份不同。结论：包 = 目录；
3. **删除重复目录**：`test/` 里的文件与 `internal/db/kv_test.go` 逐字节相同（`diff` 验证过），属于编辑器 buffer 写回来的旧副本；
4. **补全实现**：真正让 `undefined: KV` 消失的动作，是在 `internal/db/kv.go` 里把 `Get` / `Set` / `Del` 的函数体补上——因为名字最终要“存在”，才谈得上“被解析到”；
5. **四步验证**：`gofmt -l` → `go build` → `go vet` → `go test`，四个命令全绿才算完。

3.2.3. 下次照着走的排查清单

遇到 `undefined: X`，按顺序问三个问题：

问题	怎么查	典型结论
`X` 在哪里声明？	`grep -rn 'type X|func X' .`，或 `go doc <包>.X`	还没写 / 写成了别的名字 / 写成了小写（未导出）
我这个文件属于哪个包？	`go list -f '{{.ImportPath}} {{.Name}}' .`	包名与目录不一致，或文件放错了目录
两者是同一个包吗？	比对两处的 **导入路径**（不是包名）	同名不同目录 = 两个包，必须 import 且带前缀

3.3. 当前状态：你把实现清空了（练习模式）

我刚才核对时发现 `internal/db/kv.go` 已被改回练习状态——三个方法又只有签名，而且 `Open` 里的初始化也被删了：

```
func (kv *KV) Open() error {
  return nil          // ← 少了 kv.mem = map[string][]byte{}
}

func (kv *KV) Get(key []byte) (val []byte, ok bool, err error)   // ← 又只有签名
func (kv *KV) Set(key []byte, val []byte) (updated bool, err error)
func (kv *KV) Del(key []byte) (deleted bool, err error)
```

Listing 13: 当前 kv.go 的状态：回到练习模式，编译暂时不过（这是正常的，你正要自己写）

这里有个**必须知道的坑**：`Open` 里那行 map 初始化不能省。因为结构体的零值里，`mem` 是 **nil map**，而 nil map 的读写行为不对称：

```
$ go run .   # 读 nil map 的键，再往 nil map 写入
读 nil map 的键（安全，返回零值）：[]
panic: assignment to entry in nil map

goroutine 1 [running]:
main.main()
  /tmp/nilmap/main.go:8 +0xcd
```

Listing 14: 实测：读 nil map 安全，写 nil map 直接 panic

⚠ **陷阱预警** 自己实现 `Set` 时，如果 `Open` 没把 `mem` 初始化成非 nil 的 map，第一句 `kv.mem[string(key)] = val` 就会 panic——而且测试里 `Open()` 是第一步，panic 会发生在看起来最无辜的地方。正确写法：`kv.mem = map[string][]byte{}`（或 `make(map[string][]byte)`）。

3.4. 关键点

> 📌 **核心要点 报错位置告诉你“哪里在用”，不告诉你“哪里缺”。**排查 `undefined: X` 的正确顺序是：先确认报错文件 → 再确认它在哪个包 → 最后确认那个包（或它 import 的包）里到底有没有 `X` 的声明。

3.5. 变式检验

1. **换成本包内的拼写错误：** `kv := KV{}` 写成 `kv := kv{}`，报错位置相同但根因简单——`KV` 是导出类型，`kv` 是“未声明”；
2. **换成导入路径写错：** `import "github.com/zz-cpp/db-go/db"`（少了 `internal`）会报 `no required module provides package`，那是**包没找到**，与**名字没找到**是两级不同的错误；
3. **换成忘记 import：**报错会变成 `undefined: db`（包名本身没解析到），而不是 `undefined: db.KV`。

3.6. 同类归类

1. 与 C 的对照：`undefined reference to 'foo'` 是**链接期**的错误（声明有了、实现没有），Go 把这类问题提前到了编译期——同一个 `undefined: KV`，既可能是“没声明”，也可能是“不在这个包里”；
2. 与 Python 的对照：`NameError` 只有一条信息（名字没找到），而 Go 的报错带着文件路径与行列——所以**先看路径**永远是第一步；
3. 往下延伸：本课程后面会出现更多“跨层”的错误（存储层与关系层之间），同样的思路适用：看清报错属于哪一层，再去那一层找原因。

3.7. Go 语言机制补充

3.7.1. nil map 为什么读安全、写危险

map 在 Go 里是**引用类型**（底层是一个指向 hmap 结构的指针）。`var m map[K]V` 声明的变量，指针是 nil：

1. **读：** `m[k]` 在 nil map 上返回零值——实现上只是查表一次，发现没有桶就直接返回零值，无需写入；
2. **写：** `m[k] = v` 需要在哈希桶里分配空间，而 nil 指针没有 hmap 结构可操作，运行时就 panic。

`delete` 的情况同“读”——对 nil map 调用 `delete` 是安全的空操作（这也是课程里 `Del` 的实现敢直接 `delete` 的原因）。这个“读/删安全、写 panic”的不对称设计，是 Go 刻意的：它把“忘了初始化”这类 bug 尽早暴露，而不是静默地丢弃写入。

> 📌 **核心要点** 一句话收束：**遇到 `undefined`，先看路径，再看包，最后才看名字。**报错文件往往只是“受害者”，真正的缺口在声明处或包的边界上——这三步能省下大量无效修改。

4. 怎么用 go doc 查标准库的用法与实现

4.1. 费曼直觉

`go doc` 是**离线的官方文档**：它读的是你本机这套 Go 源码里的文档注释，所以它给出的答案与你的 `go1.27.1` **严格对应**——不会像搜索引擎那样混进别的版本的解释。

它有两种用途，一句话分清：**默认给“用法”，加 `-src` 给“实现”**。

4.2. 硬核机制

4.2.1. 常用形态速查

以下每一条都在本机实测过：

想看什么	命令
整包概览	`go doc strings`
整包完整文档	`go doc -all strings`（427 行，默认只有 67 行）
一行摘要列表	`go doc -short sort`
某个函数	`go doc fmt.Printf`
某个类型的定义与方法清单	`go doc strings.Builder`
类型的某个方法	`go doc strings.Builder.WriteString`
含未导出成员	`go doc -u strings.Builder`
带可执行示例	`go doc -ex strings.Builder`
实现源码	`go doc -src fmt.Printf`
浏览器里看	`go doc -http`（起一个本地文档服务器）

4.2.2. 四个实例，看清差异

```
# ① 函数文档：签名 + 说明
$ go doc fmt.Printf
func Printf(format string, a ...any) (n int, err error)
    Printf formats according to a format specifier and writes to standard
    output. It returns the number of bytes written and any write error
    encountered.

# ② 加 -src：连函数体一起给
$ go doc -src fmt.Printf
func Printf(format string, a ...any) (n int, err error) {
  return Fprintf(os.Stdout, format, a...)
}
```

Listing 15: “用法” 与 “实现” 的分界就在 -src 这一个开关

```
# ③ 结构体：类型定义 + 全部方法清单
$ go doc strings.Builder
type Builder struct {
  // Has unexported fields.
}
    A Builder is used to efficiently build a string using Builder.Write methods.
    It minimizes memory copying. The zero value is ready to use.

func (b *Builder) Cap() int
func (b *Builder) Grow(n int)
func (b *Builder) Len() int
…（共 9 个方法）

# ④ 单个方法的实现
$ go doc -src strings.Builder.WriteString
func (b *Builder) WriteString(s string) (int, error) {
  b.copyCheck()
  b.buf = append(b.buf, s...)
  return len(s), nil
}
```

Listing 16: 先看类型拿到方法清单，再逐个 -src 看实现——查一个数据结构最快的路径

📌 **核心要点** 查结构体的正确姿势是**两步**：先 `go doc <包>.<类型>` 拿到“有哪些方法”的全景，再对感兴趣的方法用 `go doc -src <包>.<类型>.<方法>` 看它怎么实现的。注意 `-src` 给的是**这个符号的声明与定义**（可能是转调别的函数），要追到底就去 `/usr/lib/go/src/` 翻全文。

4.2.3. 三种包都能查

```
# 标准库
$ go doc net/http.Client.Do

# 第三方依赖（只要已下载到模块缓存）
$ go doc github.com/stretchr/testify/assert

# 你自己的包（用相对路径）
$ go doc ./internal/db
$ go doc ./internal/db.KV
```

Listing 17: 标准库、第三方依赖、自己的包——go doc 都能查

对自己的包特别有用：`go doc ./internal/db.KV` 立刻列出 `KV` 的五个方法，等于随时确认“接口长什么样”：

```
$ go doc ./internal/db.KV
package db // import "github.com/zz-cpp/db-go/internal/db"

type KV struct {
  // Has unexported fields.
}

func (kv *KV) Close() error
func (kv *KV) Del(key []byte) (deleted bool, err error)
func (kv *KV) Get(key []byte) (val []byte, ok bool, err error)
func (kv *KV) Open() error
func (kv *KV) Set(key []byte, val []byte) (updated bool, err error)
```

Listing 18: 不用打开文件就能看到自己的接口全貌

4.3. 陷阱与易错

1. **用 `./...` 之类的模式**。官方说明写得很清楚：`The go tool's usual package mechanism does not apply: package path elements like . and ... are not implemented by go doc`——它只认真实路径，不认通配；
2. **参数以大写开头时被当成当前目录的符号**。规则是：**参数首字母大写 → 视为当前包的符号**；所以要查 `encoding/json` 就直接写包路径，查符号时才可能省略包名；
3. **`-short` 与 `-all` 打架**。两者不能组合，官方说明里明说了；
4. **以为 `-src` 能看到运行时真正执行的代码**。它给的是**该符号的源码**：`Printf` 的实现只有一行 `return Fprintf(...)`，真正的格式化逻辑在 `Fprintf` 里，需要继续往下追；
5. **拿 `go doc` 去查不存在的包**。它只能看**已安装的**包：标准库在 GOROOT，第三方要在模块缓存里（`go mod download` 之后）。

4.4. 关键点

> 📌 **核心要点** `go doc` 的两条记忆锚点：**默认 = 用法，`-src` = 实现；能查的范围 = 本机已安装的包**（标准库 + 已下载依赖 + 自己的包）。它比搜索引擎可靠的地方在于——答案与你的 Go 版本严格对应。

4.5. 变式检验

1. **换成小写参数**：`go doc json.number.int64` 与 `go doc json.Number.Int64` 输出完全相同——规则是“小写匹配任意大小写，大写必须精确匹配”；
2. **换成查看工具自身**：`go doc cmd/doc` 能看 `go doc` 这个工具包的文档；`go doc -cmd cmd/doc` 连它的导出符号一起列出来；
3. **换成看整体设计**：`go doc -all <包>` 会把包注释（通常写着“这个包解决什么问题”）放在最前面，比逐个符号查更能建立全局观。

4.6. 同类归类

1. 与 `go help` 的分工：`go doc` 答“这个 API 怎么用”，`go help` 答“这个命令怎么用”；一个针对**包与符号**，一个针对**工具链**；

2. 与 pkg.go.dev 的关系：同一个数据源（源码注释），一个在线一个离线。断开网络或想确认本机版本行为时用 `go doc`；
3. 往下延伸：本课程后面要读的 `log`、`os`、`encoding/binary`、`sort` 这些标准库包，都可以先用 `go doc -all <包>` 扫一遍包注释，再对着 `-src` 啃实现——这比直接读源码省力得多。

4.7. Go 语言机制补充

4.7.1. 文档就是源码注释

`go doc` 输出的那些说明文字，全部来自源码里的注释——紧挨着声明、且与声明之间没有空行的那种注释（这叫 doc comment）。例如：

```
// Printf formats according to a format specifier and writes to standard output.
// It returns the number of bytes written and any write error encountered.
func Printf(format string, a ...any) (n int, err error) {
```

Listing 19: 文档注释紧跟声明，中间不能有空行；这就是 go doc 的全部信息来源

这条机制有实际后果：**你给自己的包写 doc comment，别人（和未来的你）就能用 `go doc` 查到**。写库时“文档”不是额外工作，就是把注释放在对的位置。课程解答里的 `db0101` 包没写注释，所以 `go doc` 只能列出签名——这也是为什么本项目的包值得在实现时补上注释。

4.7.2. `go doc` 背后的实现

`go doc` 本身也是一个包（`cmd/doc`），它做的事是：解析目标包的源码 → 提取 doc comment → 按 `go doc` 的规则格式化输出。所以它**不需要联网、不需要启动服务**，也**不会给你过期的文档**——注释改了，输出立刻跟着变。

> 📌 **核心要点** 一句话收束：**`go doc` 是“离线且版本精确的官方文档”，`-src` 把它变成“源码浏览器”**。 遇到不熟的 API 先 `go doc`，遇到不熟的实现再 `-src`，两者配合基本可以替代大部分搜索。

Index of Listings

Listing 1 同一份输出里两个包的 Name 都是 db，但 ImportPath 不同——它们就是两个包 4
Listing 2 报错文件是 `test/kv_test.go` —— 不是 `internal/db/kv_test.go` 4
Listing 3 同机实测：放对目录的那一份，gopls 一个字都不报 4
Listing 4 探针：在 test/ 目录里引用 KV 5
Listing 5 照样 undefined——因为那个目录下根本没有 KV 的声明 5
Listing 6 当前唯一阻塞：kv.go 里 Get / Set / Del 三个方法还只有签名 7
Listing 7 讲义目录结构：既有课程正文，也有每一步的参考解答 8
Listing 8 课程作者的 0101 实现—— `updated` 的真实语义在这里：值真的变了才为 true 8
Listing 9 补全后的 kv.go：新增 import “bytes”，三个方法各就各位 9
Listing 10 四个命令全部通过，测试转绿 10
Listing 11 命名返回值让“给哪个变量赋值”一目了然，裸 return 省去重复列举 11
Listing 12 第 1 步的 KV 完成，四步全绿 12
Listing 13 当前 kv.go 的状态：回到练习模式，编译暂时不过（这是正常的，你正要自己写） 14
Listing 14 实测：读 nil map 安全，写 nil map 直接 panic 14
Listing 15 “用法” 与 “实现” 的分界就在 -src 这一个开关 16
Listing 16 先看类型拿到方法清单，再逐个 -src 看实现——查一个数据结构最快的路径 17
Listing 17 标准库、第三方依赖、自己的包——go doc 都能查 17
Listing 18 不用打开文件就能看到自己的接口全貌 18
Listing 19 文档注释紧跟声明，中间不能有空行；这就是 go doc 的全部信息来源 19